

Infraestrutura como Código para Ambientes Linux: Estudo Comparativo entre Ansible com apoio de IA e *Deploy Manual*

Carlos Henrique Cerqueira Santos¹

¹Curso de Sistemas de Informação
Centro Universitário Jorge Amado – UNIJORGE
Salvador – BA – Brasil

henriquecerqueira789@gmail.com

Abstract. *This work compares Infrastructure as Code (IaC) automation via Ansible against manual deployment for provisioning Linux server environments. Experiments were conducted in a VMware-hosted lab with three Ubuntu 22.04 LTS virtual machines interconnected by VPN and SSH key authentication. Ansible provisioned two servers simultaneously in 1 minute and 16 seconds, against 30 minutes and 50 seconds for manual single-server deployment, yielding a 24:1 time ratio. Configuration standardization was confirmed by identical PLAY RECAP counters across nodes (H1), and functional idempotency by a `changed=5` result on the second run (H2). A large language model supported playbook development by diagnosing MySQL 8.0 authentication incompatibilities and suggesting security improvements, though all suggestions required technical review before application (H3).*

Resumo. *Este trabalho compara a automação por Infraestrutura como Código (IaC) via Ansible com o deploy manual no provisionamento de ambientes Linux. Os experimentos foram conduzidos em laboratório virtualizado no VMware, com três máquinas virtuais Ubuntu 22.04 LTS interligadas por VPN e acesso SSH por chave criptográfica. O Ansible provisionou dois servidores simultaneamente em 1 minuto e 16 segundos, contra 30 minutos e 50 segundos no processo manual para um único servidor, razão de 24:1 em favor da automação. A padronização foi confirmada por contadores idênticos no PLAY RECAP entre os nós (H1), e a idempotência funcional pelo resultado `changed=5` na segunda execução (H2). O DeepSeek apoiou o desenvolvimento dos playbooks, diagnosticando incompatibilidades de autenticação do MySQL 8.0 e sugerindo melhorias de segurança, com revisão técnica obrigatória antes de cada aplicação (H3).*

1. Introdução

A expansão das infraestruturas digitais nas últimas décadas impôs desafios crescentes à administração de servidores: ambientes que antes eram gerenciados por rotinas manuais bem definidas passaram a exigir consistência entre dezenas ou centenas de máquinas, reprodutibilidade de configurações e rastreabilidade de mudanças. Distribuições Linux consolidaram-se como base dessas infraestruturas pela estabilidade e pelo custo operacional reduzido [Red Hat 2022], mas a configuração manual de cada servidor introduz variações sutis que, acumuladas, comprometem a previsibilidade do ambiente — fenômeno conhecido como *configuration drift* [Morris 2016].

A Infraestrutura como Código (*Infrastructure as Code* – IaC) responde a esse problema ao tratar configurações de servidores como artefatos de software: versionáveis, testáveis e aplicáveis de forma repetível [Morris 2016]. O Ansible, ferramenta de automação *agentless* mantida pela Red Hat, opera via SSH sem exigir agentes nos nós gerenciados, o que o torna especialmente adequado a ambientes Linux heterogêneos [Maya 2020]. Sua adoção permite que o estado desejado de múltiplos servidores seja declarado em arquivos YAML e aplicado simultaneamente, eliminando a variabilidade introduzida pelo fator humano.

Mais recentemente, modelos de linguagem de grande escala (LLMs) têm sido utilizados como ferramentas de apoio na geração e depuração de scripts de automação [Srivatsa et al. 2024]. Esse uso, contudo, não é isento de limitações: estudos indicam que os modelos tendem a falhar em casos envolvendo versões específicas de ferramentas ou requisitos de idempotência mais complexos [Srivatsa et al. 2024], o que torna indispensável a revisão técnica antes de qualquer aplicação em produção.

Este trabalho analisa, em ambiente virtualizado controlado, o provisionamento de servidores Linux via Ansible comparado ao processo manual equivalente, documentando diferenças de tempo, padronização e consistência. O uso de IA como ferramenta de apoio ao desenvolvimento dos *playbooks* é igualmente registrado e avaliado. O estudo busca responder em que medida a IaC com Ansible reduz o tempo e aumenta a padronização do provisionamento, e em que condições a IA contribui para a qualidade dos scripts produzidos.

2. Definição do Projeto

2.1. Tema

Infraestrutura como Código para ambientes Linux: um estudo comparativo entre Ansible com apoio de IA e *deploy* manual.

2.2. Justificativa

A administração manual de servidores Linux em ambientes de múltiplos nós produz, com frequência, divergências de configuração entre máquinas que deveriam ser idênticas. Versões de pacotes instaladas em momentos distintos, permissões ajustadas pontualmente e serviços reiniciados com parâmetros diferentes acumulam-se silenciosamente até comprometer a previsibilidade do ambiente [Red Hat 2022]. Esse problema se agrava na medida em que o número de servidores cresce, pois o esforço de manutenção cresce proporcionalmente e a probabilidade de erro humano aumenta a cada intervenção.

A IaC, e em particular o Ansible, endereça esse problema ao tornar o estado desejado da infraestrutura explícito em arquivos YAML versionáveis, aplicáveis de forma idempotente em quantos nós forem necessários [Maya 2020]. Trabalhos anteriores documentam reduções de tempo de restauração e de incidência de falhas por configuração incorreta com a adoção de IaC [Alves and Ramos 2020], mas estudos comparativos com medição direta de tempo em ambiente controlado ainda são escassos na literatura nacional. Este trabalho preenche essa lacuna ao conduzir o experimento em laboratório virtualizado reproduzível, com cronometragem explícita de ambos os processos e registro sistemático do uso de IA como ferramenta de apoio.

2.3. Problema de Pesquisa

Em que medida o provisionamento de servidores Linux via Ansible reduz o tempo de *deploy* e aumenta a consistência das configurações em comparação ao processo manual, e em que condições o apoio de Inteligência Artificial contribui para a qualidade dos scripts de automação produzidos?

2.4. Hipóteses

H1 – A utilização da Infraestrutura como Código por meio da ferramenta Ansible contribui para maior padronização e consistência na configuração de servidores Linux quando comparada ao processo tradicional de *deploy* manual.

H2 – O provisionamento automatizado via Ansible é significativamente mais rápido que o processo manual equivalente, considerando o mesmo conjunto de pacotes e serviços instalados no mesmo sistema operacional.

H3 – O uso de Inteligência Artificial como ferramenta de apoio ao desenvolvimento de *playbooks* Ansible contribui para o diagnóstico de incompatibilidades e para a melhoria da segurança do código produzido, sendo necessária revisão técnica antes de qualquer aplicação em produção.

3. Objetivos

3.1. Objetivo Geral

Comparar o provisionamento de ambientes Linux via Ansible com o *deploy* manual equivalente, avaliando diferenças de tempo, padronização e consistência de configurações, e documentar a contribuição do apoio de Inteligência Artificial no desenvolvimento dos *playbooks* produzidos.

3.2. Objetivos Específicos

- Estruturar um ambiente experimental virtualizado com três servidores Linux para viabilizar a execução controlada e reproduzível dos experimentos comparativos;
- Desenvolver *playbooks* Ansible para o provisionamento automatizado da pilha LAMP, contemplando instalação de pacotes, configuração de serviços e validação de estados;
- Executar o *deploy* manual equivalente no mesmo ambiente, documentando cada etapa, o tempo total de execução e as interrupções interativas ocorridas;
- Comparar os resultados de ambos os processos quanto à padronização entre nós, ao tempo de execução e à idempotência das configurações, verificando H1 e H2;
- Registrar e analisar as interações com o modelo de IA durante o desenvolvimento dos *playbooks*, identificando os tipos de contribuição oferecidos e os limites de aplicação sem revisão técnica, verificando H3.

4. Embasamento Teórico

4.1. Infraestrutura como Código

A Infraestrutura como Código (*Infrastructure as Code* – IaC) consiste na aplicação de práticas do desenvolvimento de software ao gerenciamento e provisionamento de infraestrutura computacional [Morris 2016]. Configurações de servidores, redes e serviços

são descritas em arquivos versionáveis e reutilizáveis, substituindo intervenções manuais sujeitas a erros e inconsistências [Red Hat 2022].

Diferentemente da administração tradicional, na qual cada servidor pode divergir sutilmente dos demais ao longo do tempo, a IaC trata a infraestrutura como artefato de software: passível de versionamento, revisão por pares, testes automatizados e *rollback* [Morris 2016]. Essa mudança de paradigma viabiliza a reprodutibilidade de ambientes e a rastreabilidade de mudanças [Lopes 2019]. Estudos práticos documentam reduções no tempo de restauração de ambientes e na incidência de falhas por configuração incorreta após a adoção de IaC [Alves and Ramos 2020].

4.2. DevOps e Cultura de Automação

DevOps reúne práticas, princípios culturais e ferramentas que aproximam as equipes de desenvolvimento (*Development*) e operações (*Operations*), com o objetivo de encurtar o ciclo de entrega de software e aumentar a confiabilidade dos sistemas em produção [Kim et al. 2018]. A automação de infraestrutura ocupa papel central nessa cultura: tarefas manuais repetitivas introduzem variabilidade que, eliminada pela automação, acelera o provisionamento e reduz a taxa de falhas [Matsui and Goya 2021].

A IaC é uma expressão direta desse princípio. Ferramentas como o Ansible permitem que o estado desejado de um ambiente seja declarado em arquivos YAML e aplicado de forma consistente em múltiplos servidores simultaneamente [Red Hat 2022], tratando infraestrutura com o mesmo rigor aplicado ao código de aplicação: versionada, testada e implantada de forma automatizada [Kim et al. 2018].

4.3. Gerenciamento de Configuração e *Configuration Drift*

O gerenciamento de configuração consiste no conjunto de práticas voltadas ao controle e à manutenção do estado dos sistemas computacionais ao longo do tempo [Morris 2016]. Em ambientes administrados manualmente, cada intervenção humana representa uma oportunidade para que o ambiente se desvie de seu estado original, sem que esse desvio seja necessariamente documentado ou revertível.

Esse fenômeno é conhecido como *configuration drift*: a divergência progressiva entre o estado real de um sistema e o estado esperado ou documentado [Red Hat 2022]. Com o tempo, servidores que deveriam ser idênticos passam a apresentar diferenças sutis em versões de pacotes, permissões de arquivos ou configurações de serviços, tornando o ambiente imprevisível e difícil de reproduzir [Alves and Ramos 2020]. A IaC endereça diretamente esse problema ao tornar o estado desejado da infraestrutura explícito, versionado e automaticamente aplicável [Morris 2016]. Ferramentas como o Ansible eliminam o *drift* ao garantir que cada execução do *playbook* reconcilie o estado atual do sistema com o estado declarado, independentemente das intervenções manuais que possam ter ocorrido entre execuções [Maya 2020].

4.4. Idempotência e Consistência de Estado

Idempotência é a propriedade pela qual a execução repetida de uma operação produz sempre o mesmo resultado final, independentemente do estado inicial do sistema [Morris 2016]. Na prática, um *playbook* Ansible pode ser executado em um servidor já configurado sem introduzir alterações indesejadas: apenas as divergências em relação ao

estado declarado são corrigidas. Pesquisas sobre métricas de qualidade em *playbooks* identificam a idempotência como um dos principais indicadores de maturidade de um script de automação [Palma et al. 2020].

No contexto deste trabalho, a idempotência é verificada pelo contador `changed` no PLAY RECAP. Na segunda execução do *playbook*, cinco tarefas reportaram `changed` em ambos os nós — não por alterações reais de configuração, mas porque utilizam `failed_when` e `changed_when` baseados em `stdout`, padrão necessário para contornar a autenticação *socket* do MySQL 8.0. Nenhuma alteração efetiva de estado foi introduzida, o que caracteriza idempotência funcional, distinta da idempotência absoluta (`changed=0`).

4.5. Ansible: Arquitetura e Funcionamento

O Ansible é uma ferramenta de automação de código aberto, mantida pela Red Hat, que permite o gerenciamento de configurações, o provisionamento de servidores e a orquestração de tarefas por meio de *playbooks* escritos em YAML [Red Hat 2022]. Sua arquitetura *agentless* — que dispensa a instalação de agentes nos nós gerenciados — e a comunicação via SSH tornam-no especialmente adequado a ambientes Linux, reduzindo a complexidade de implantação e manutenção da própria ferramenta [Maya 2020].

O Ansible organiza os servidores em um arquivo de inventário, estático ou dinâmico, e aplica as tarefas dos *playbooks* de forma sequencial e declarativa. O uso de *roles* modulariza e reutiliza configurações; o Ansible Vault permite o armazenamento cifrado de credenciais e variáveis sensíveis [Ansible Community 2026]. Sua adoção em ambientes de redes abertas para gerenciamento em larga escala evidencia versatilidade além do provisionamento básico [Salazar-Chacón and Parra 2023].

4.6. Comparação entre Ferramentas de IaC

O ecossistema de Infraestrutura como Código conta com diversas ferramentas, cada uma com características distintas de arquitetura, linguagem e modelo de operação [da Silva Júnior 2022]. A Tabela 1 apresenta uma comparação entre as principais soluções disponíveis.

Tabela 1. Comparação entre principais ferramentas de IaC

Ferramenta	Linguagem	Agente	Foco principal	Aprendizado
Ansible	YAML	Não	Conf. e orquestração	Baixo
Puppet	DSL própria	Sim	Ger. de configuração	Alto
Chef	Ruby/DSL	Sim	Ger. de configuração	Alto
Terraform	HCL	Não	Provis. em nuvem	Médio
SaltStack	YAML/Python	Sim	Conf. e automação	Médio

Fonte: elaborado pelo autor com base em [da Silva Júnior 2022] e [Morris 2016] (2026)

A escolha do Ansible justifica-se pela arquitetura *agentless*, pela baixa curva de aprendizado, pela integração nativa com SSH e pela ampla adoção em ambientes Linux [Maya 2020]. Diferentemente do Terraform, cujo foco é o provisionamento de recursos em nuvem, o Ansible destaca-se no gerenciamento de configurações de servidores existentes [da Silva Júnior 2022] — escopo diretamente alinhado ao deste trabalho.

4.7. Virtualização como Ambiente de Laboratório

A virtualização consiste na criação de instâncias isoladas de recursos computacionais sobre um único equipamento físico por meio de um *hypervisor*, viabilizando laboratórios controlados, reproduzíveis e de baixo custo para validação de arquiteturas e ferramentas [Calderón et al. 2022].

Neste trabalho, o VMware Workstation hospedou três máquinas virtuais Ubuntu 22.04.5 LTS sobre um servidor HP ProLiant ML350 Gen10 com 10 núcleos e 160 GB de RAM. O isolamento provido pela virtualização garantiu que variáveis externas — carga de rede, estado de disco, interferência de outros processos — não afetassem os tempos medidos, requisito essencial para a validade comparativa dos experimentos. Abordagem equivalente é adotada em trabalhos acadêmicos similares como alternativa válida à infraestrutura de nuvem pública [Lopes 2019].

4.8. Inteligência Artificial no Desenvolvimento de Scripts de Automação

Modelos de linguagem de grande escala (LLMs) têm sido aplicados como ferramentas de apoio à geração e depuração de scripts de infraestrutura. Estudo recente demonstra desempenho promissor na criação de *playbooks* Ansible e arquivos Terraform a partir de descrições em linguagem natural, com capacidade de reduzir erros de sintaxe e sugerir boas práticas de segurança [Srivatsa et al. 2024]. O mesmo estudo identifica limitações consistentes: os modelos tendem a falhar em casos que envolvem versões específicas de ferramentas, módulos menos documentados ou requisitos de idempotência mais complexos [Srivatsa et al. 2024].

No contexto da IaC, a IA não substitui o conhecimento técnico do administrador, mas atua como ferramenta de diagnóstico e orientação [Veiga et al. 2025]. Neste trabalho, o modelo foi acionado pontualmente para diagnosticar a incompatibilidade de autenticação do MySQL 8.0 e sugerir o parâmetro `validate` no arquivo `sudoers`. Em ambos os casos, as sugestões exigiram revisão técnica antes da aplicação — comportamento consistente com as limitações identificadas por [Srivatsa et al. 2024] e com o escopo de H3.

5. Metodologia

5.1. Sujeito da Pesquisa

O sujeito desta pesquisa é o ambiente computacional virtualizado utilizado nos experimentos, composto por três máquinas virtuais com Ubuntu 22.04.5 LTS hospedadas no VMware Workstation. Os dados coletados derivam exclusivamente do comportamento dos sistemas durante a execução dos processos de provisionamento automatizado e manual.

As máquinas desempenham os seguintes papéis: *ansible-controller*, responsável pela execução dos *playbooks* e pelo controle da automação; *web01* e *web02*, nós gerenciados que recebem o provisionamento da pilha LAMP (Linux, Apache, MySQL, PHP). A comunicação entre as máquinas foi estabelecida via VPN (WireGuard), com acesso SSH por chave criptográfica Ed25519. As especificações estão descritas na Tabela 2.

 [Em teste] ansible-controller	 Powered on	 Normal	Ubuntu Linux
 [Em teste] web01	 Powered on	 Normal	Ubuntu Linux
 [Em teste] web02	 Powered on	 Normal	Ubuntu Linux

Figura 1. Ambiente virtualizado com as três máquinas virtuais em execução
 Fonte: elaborado pelo autor (2026)

Tabela 2. Especificações do ambiente experimental

VM	Função	CPU	RAM	IP
ansible-controller	Controlador Ansible	1 vCPU	1,9 GiB	172.16.101.5
web01	Nó gerenciado	1 vCPU	1,9 GiB	172.16.100.26
web02	Nó gerenciado	1 vCPU	1,9 GiB	172.16.100.27

Fonte: elaborado pelo autor (2026)

```

Nova aba  Dividir a exibição
hcerqueira@ansible-controller:~/ansible-tcc$ sudo ./dadosvm.sh
--- HOSTNAME: ansible-controller
--- CPU: 1 vCPU (Intel Xeon Silver 4210R @ 2.40GHz)
--- RAM: Total: 1.9Gi | Used: 194Mi | Available: 1.5Gi
--- IP (IPv4): 172.16.101.5
--- OS: Ubuntu 22.04.5 LTS
--- Disco: Size: 20G | Used: 6.8G | Use%: 37%
--- Ansible: 2.10.8
  
```

Figura 2. Especificações da máquina *ansible-controller*
 Fonte: elaborado pelo autor (2026)

```

Nova aba  Dividir a exibição
hcerqueira@web01:~$ sudo ./dadosvm.sh
--- HOSTNAME: web01
--- CPU: 1 vCPU (Intel Xeon Silver 4210R @ 2.40GHz)
--- RAM: Total: 1.9Gi | Used: 199Mi | Available: 1.5Gi
--- IP (IPv4): 172.16.100.26
--- OS: Ubuntu 22.04.5 LTS
--- Disco: Size: 15G | Used: 5.7G | Use%: 41%
--- Ansible: Nao instalado (agentless)
  
```

Figura 3. Especificações da máquina *web01*
 Fonte: elaborado pelo autor (2026)

```

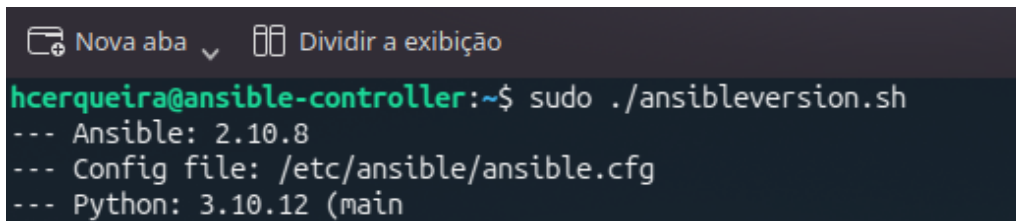
Nova aba  Dividir a exibição
hcerqueira@web02:~$ sudo ./dadosvm.sh
--- HOSTNAME: web02
--- CPU: 1 vCPU (Intel Xeon Silver 4210R @ 2.40GHz)
--- RAM: Total: 1.9Gi | Used: 199Mi | Available: 1.5Gi
--- IP (IPv4): 172.16.100.27
--- OS: Ubuntu 22.04.5 LTS
--- Disco: Size: 15G | Used: 5.7G | Use%: 41%
--- Ansible: Nao instalado (agentless)
  
```

Figura 4. Especificações da máquina *web02*
 Fonte: elaborado pelo autor (2026)

5.2. Instrumento de Coleta de Dados

Os dados foram coletados por meio de quatro instrumentos:

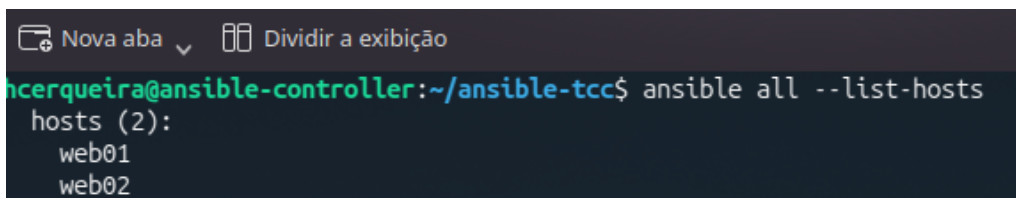
- **Logs do Ansible:** saída completa do comando `ansible-playbook`, capturada com `tee`, contendo os contadores do PLAY RECAP (ok, changed, failed) para cada nó gerenciado;
- **Cronometragem automatizada:** tempo de execução do *playbook* capturado com `time ansible-playbook site.yml`, registrando os tempos *real*, *user* e *sys*;
- **Cronometragem do *deploy manual*:** tempo total do processo de configuração em *web01*, registrado com o comando `date` no início e no fim, com cada etapa executada individualmente e sem automação;
- **Registro das interações com IA:** capturas de tela das sessões com o DeepSeek, documentando os problemas apresentados, as sugestões geradas e as correções aplicadas aos *playbooks* — instrumento qualitativo para verificação de H3.



```
Nova aba  Dividir a exibição
hcerqueira@ansible-controller:~$ sudo ./ansibleversion.sh
--- Ansible: 2.10.8
--- Config file: /etc/ansible/ansible.cfg
--- Python: 3.10.12 (main)
```

Figura 5. Versão do Ansible e especificações do nó controlador

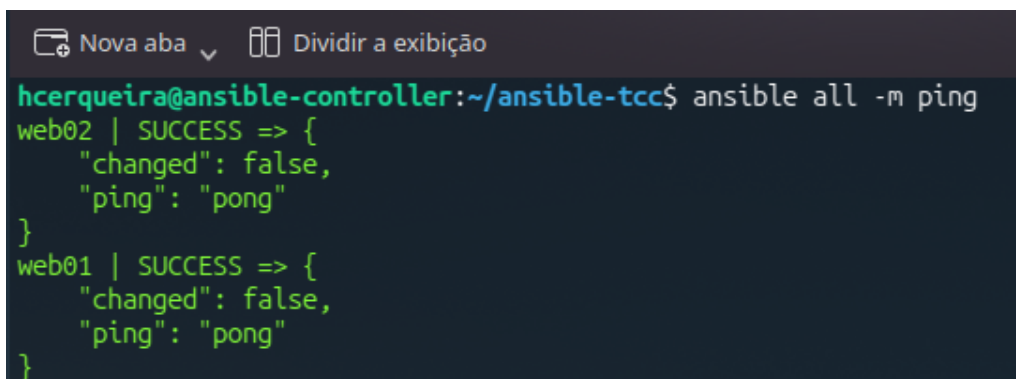
Fonte: elaborado pelo autor (2026)



```
Nova aba  Dividir a exibição
hcerqueira@ansible-controller:~/ansible-tcc$ ansible all --list-hosts
hosts (2):
  web01
  web02
```

Figura 6. Listagem de hosts reconhecidos pelo inventário Ansible

Fonte: elaborado pelo autor (2026)



```
Nova aba  Dividir a exibição
hcerqueira@ansible-controller:~/ansible-tcc$ ansible all -m ping
web02 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
web01 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
```

Figura 7. Verificação de conectividade com `ansible all -m ping`

Fonte: elaborado pelo autor (2026)

5.3. Corpus

O corpus deste estudo é composto por três conjuntos de dados coletados durante os experimentos. O primeiro conjunto abrange os contadores do PLAY RECAP das execuções em modo *check* (*dry run*), primeira execução (run1) e segunda execução (run2), utilizados para verificação de H1. O segundo conjunto compreende o tempo de execução do *playbook* Ansible e o tempo cronometrado do *deploy* manual, ambos em segundos, utilizados para verificação de H2. O terceiro conjunto reúne os registros qualitativos das interações com a IA — capturas de tela, diagnósticos e correções aplicadas — utilizados para verificação de H3.

5.4. Análise de Dados

A análise seguiu abordagem mista: quantitativa para H1 e H2, e qualitativa para H3.

Para **H1**, os contadores do PLAY RECAP de *web01* e *web02* são comparados entre si e entre execuções. A padronização é verificada pela equivalência dos contadores entre nós; a idempotência funcional, pelo resultado `changed=5` na segunda execução — valor decorrente do comportamento de `changed_when` nas tarefas de autenticação do MySQL 8.0, sem alteração efetiva de estado.

Para **H2**, o tempo total do *deploy* manual em um único servidor é comparado ao tempo da execução automatizada em dois servidores simultâneos. A análise normaliza os valores por servidor para produzir a razão de tempo entre os dois processos.

Para **H3**, os registros das interações com a IA são examinados qualitativamente para identificar os tipos de contribuição oferecidos — diagnóstico de incompatibilidade, sugestão de código, melhoria de segurança — e os casos em que a revisão técnica foi necessária antes da aplicação.

5.5. Artefatos de Automação

O provisionamento automatizado foi implementado por meio de uma *role* Ansible denominada `lamp`, estruturada em tarefas modulares. Os Quadros 3 a 9 descrevem os principais artefatos desenvolvidos.

Tabela 3. Inventário Ansible – `hosts.ini`

```
[webservers]
web01  ansible_host=172.16.100.26  ansible_user=hcerqueira
       ansible_ssh_private_key_file=~/.ssh/id_ed25519
web02  ansible_host=172.16.100.27  ansible_user=hcerqueira
       ansible_ssh_private_key_file=~/.ssh/id_ed25519

[webservers:vars]
ansible_python_interpreter=/usr/bin/python3
```

Fonte: elaborado pelo autor (2026)

A Tabela 10 descreve as etapas executadas durante o *deploy* manual, utilizado como linha de base para H2, e a Tabela 11 lista os pacotes instalados em cada etapa.

Tabela 4. Configuração do ambiente Ansible – ansible.cfg

```
[defaults]
inventory          = inventory/hosts.ini
remote_user        = hcerqueira
private_key_file    = ~/.ssh/id_ed25519
host_key_checking   = False
stdout_callback     = yaml
vault_password_file = .vault_pass

[privilege_escalation]
become              = True
become_method       = sudo
become_user         = root
become_ask_pass     = False
```

Fonte: elaborado pelo autor (2026)

Tabela 5. *Playbook* principal de provisionamento – site.yml

```
- name: Provisionar stack LAMP
  hosts: webserver
  become: yes
  gather_facts: yes

  pre_tasks:
    - name: Verificar conectividade
      ping:
    - name: Registrar inicio
      debug:
        msg: "Iniciando em {{ inventory_hostname }} -
              {{ ansible_date_time.iso8601 }}"

  roles:
    - lamp

  post_tasks:
    - name: Verificar Apache na porta 80
      wait_for:
        host: "{{ ansible_default_ipv4.address }}"
        port: 80
        timeout: 15
    - name: Registrar conclusao
      debug:
        msg: "Concluido em {{ inventory_hostname }}"
```

Fonte: elaborado pelo autor (2026)

Tabela 6. Organização de tarefas da *role* LAMP – tasks/main.yml

```
- import_tasks: users.yml      # criacao do usuario de deploy
  tags: users
- import_tasks: packages.yml  # instalacao de pacotes base
  tags: packages
- import_tasks: apache.yml    # configuracao do Apache
  tags: apache
- import_tasks: mysql.yml     # configuracao do MySQL
  tags: mysql
- import_tasks: firewall.yml  # regras UFW
  tags: firewall
- import_tasks: deploy.yml    # deploy da aplicacao PHP
  tags: deploy
```

Fonte: elaborado pelo autor (2026)

Tabela 7. Instalação de pacotes da pilha LAMP – packages.yml

```
- name: Instalar pacotes base da pilha LAMP
  apt:
    name:
      - apache2
      - mysql-server
      - python3-pymysql  # dependencia para modulos community.mysql
      - php
      - php-mysql
      - php-cli
      - libapache2-mod-php
    state: present
    update_cache: yes
```

Fonte: elaborado pelo autor (2026)

Tabela 8. Configuração de *firewall* com *rate limiting* no SSH – firewall.yml

```
# ufw limit: protecao contra forza bruta no SSH
- name: UFW - limitar SSH (rate limiting)
  ufw:
    rule: limit
    port: "22"
    proto: tcp

- name: UFW - permitir portas HTTP/HTTPS
  ufw:
    rule: allow
    port: "{{ item.split('/')[0] }}"
    proto: "{{ item.split('/')[1] }}"
    loop: "{{ ufw_allowed_ports }}"  # [80/tcp, 443/tcp]

- name: Habilitar UFW
  ufw:
    state: enabled
```

Fonte: elaborado pelo autor com apoio do DeepSeek (2026)

Tabela 9. Proteção de credenciais com Ansible Vault – group_vars/all/main.yml

```
# Variaveis publicas referenciam variaveis cifradas pelo Vault.
# As senhas reais ficam em vault.yml (criptografado AES256).
mysql_root_password: "{{ vault_mysql_root_password }}"
mysql_db_password:   "{{ vault_mysql_db_password }}"

mysql_db_name: appdb
mysql_db_user: appuser
```

Fonte: elaborado pelo autor (2026)

Tabela 10. Etapas do *deploy* manual – Ubuntu 22.04 (H2)

Etapa	Descrição	Tempo real
1	Atualizar cache de pacotes (apt update)	~1 min
2	Instalar Apache (apt install apache2)	~2 min
3	Instalar MySQL (apt install mysql-server)	~3 min
4	Configurar autenticação MySQL 8.0	~2 min
5	Criar banco de dados e usuário (SQL)	~2 min
6	Instalar PHP e extensões	~3 min
7	Configurar VirtualHost do Apache	~1 min
8	Criar usuário <i>deployer</i> e permissões	~1 min
9	Configurar regras de <i>firewall</i> (UFW) *interativo*	~2 min
10	Fazer <i>deploy</i> da aplicação PHP	~1 min
11	Reiniciar serviços e verificar portas	~1 min
12	Validar aplicação	~1 min
Total (um servidor)		30min 50s

Fonte: elaborado pelo autor (2026)

Tabela 11. Pacotes instalados durante o *deploy* manual

Etapa	Pacote	Função
2	apache2	Servidor web HTTP
3	mysql-server	Sistema gerenciador de banco de dados
6	php	Interpretador PHP
6	php-mysql	Extensão PHP para MySQL
6	php-cli	Interface de linha de comando PHP
6	libapache2-mod-php	Módulo PHP para Apache
9	ufw	<i>Firewall</i> com interface simplificada

Fonte: elaborado pelo autor (2026)

6. Recursos

Os recursos utilizados neste trabalho abrangem três categorias: hardware físico, que hospeda o ambiente virtualizado; software de automação, gerenciamento e editoração; e serviços de apoio ao desenvolvimento e versionamento. A seleção priorizou ferramen-

tas de código aberto ou gratuitas, compatíveis com o ambiente Ubuntu 22.04 LTS e adequadas à reprodução do experimento em infraestrutura local. A Tabela 12 detalha cada recurso com sua versão e função no contexto do estudo.

Tabela 12. Recursos utilizados no experimento

Categoria	Recurso	Descrição
Hardware (host)	HP ProLiant ML350 Gen10	Xeon Silver 4210R, 10 núcleos, 160GB RAM
Hipervisor	VMware Workstation 17	3 VMs Ubuntu 22.04 LTS
SO das VMs	Ubuntu 22.04.5 LTS	Kernel 5.15.0-181-generic (três VMs)
Automação (IaC)	Ansible 2.10.8	Agentless, via SSH, conf. <code>ansible.cfg</code>
Banco de dados	MySQL Server 8.0.41	Auth. <code>mysql_native_password</code>
Servidor web	Apache 2.4.52	PHP 8.1.2 (<code>libapache2-mod-php</code>)
Firewall	UFW 0.36	Rate limit SSH + Ansible Vault
IA de apoio	DeepSeek	Diagnóstico e sugestões nos <i>playbooks</i>
Rede VPN	WireGuard	Conectividade entre controlador e nós
Versionamento	GitHub	<code>unijorge/tcc-ansible-iac</code>
Editoração	Overleaf + SBC template	LaTeX com <code>sbc-template.sty</code>

Fonte: elaborado pelo autor (2026)

7. Cronograma

A Tabela 13 apresenta o cronograma de execução deste trabalho em formato de Diagrama de Gantt, distribuído em oito semanas, com a carga horária estimada por atividade.

Tabela 13. Cronograma de atividades – Diagrama de Gantt

Atividade	S1	S2	S3	S4	S5	S6	S7	S8	CH
Configuração do ambiente VMware (VMs, rede, VPN, SSH)	•								8h
Desenvolvimento dos <i>playbooks</i> Ansible (<i>role</i> LAMP, Vault)		•	•						20h
Apoio de IA: diagnóstico e refinamento dos <i>playbooks</i>			•						6h
<i>Deploy</i> manual em <i>web01</i> : cronometragem e documentação				•					6h
Provisionamento automatizado: <i>dry run</i> , <i>run1</i> , <i>run2</i> e coleta de dados				•					4h
Redação do TCC no Overleaf (todas as seções)					•	•	•		16h
Revisão final, formatação LaTeX e submissão								•	8h
Total									68h

Fonte: elaborado pelo autor (2026)

8. Resultados

8.1. H1 — Padronização e Idempotência

A execução em modo *check* (*dry run*) produziu contadores idênticos em ambos os nós, confirmando que o *playbook* aplicaria as mesmas alterações independentemente do nó

gerenciado (Tabela 14).

Tabela 14. Execução em modo *check* – padronização (H1)

Nó	ok	changed	failed	skipped
web01	27	3	0	13
web02	27	3	0	13

Fonte: saída do terminal – elaborado pelo autor (2026)

A primeira execução completa (run1) provisionou *web01* e *web02* com resultados idênticos: 41 tarefas executadas com sucesso e 19 alterações aplicadas em cada nó, sem falhas (Tabela 15).

Tabela 15. Primeira execução completa – run1 (H1)

Nó	ok	changed	failed	skipped
web01	41	19	0	0
web02	41	19	0	0

Fonte: saída do terminal – elaborado pelo autor (2026)

```
PLAY RECAP *****
*****
web01                : ok=41  changed=19  unreachable=0
  failed=0    skipped=0    rescued=0    ignored=0
web02                : ok=41  changed=19  unreachable=0
  failed=0    skipped=0    rescued=0    ignored=0

hcerqueira@ansible-controller:~/ansible-tcc$
```

Figura 8. PLAY RECAP da primeira execução completa do *playbook*

Fonte: elaborado pelo autor (2026)

Na segunda execução, realizada sem qualquer alteração no ambiente, ambos os nós reportaram *changed=5* em 29,9 segundos (Tabela 16). As cinco tarefas sinalizadas como alteradas correspondem à configuração de autenticação do MySQL 8.0, que utiliza *changed_when* baseado em *stdout* para contornar o *plugin auth_socket* — comportamento esperado e sem efeito real sobre o estado do sistema, caracterizando idempotência funcional.

Tabela 16. Segunda execução – verificação de estado (H1)

Nó	ok	changed	failed	skipped
web01	40	5	0	0
web02	40	5	0	0

Fonte: saída do terminal – elaborado pelo autor (2026)

```
PLAY RECAP *****
*****
web01      : ok=40    changed=5    unreachable=0
failed=0    skipped=0    rescued=0    ignored=0
web02      : ok=40    changed=5    unreachable=0
failed=0    skipped=0    rescued=0    ignored=0

real    0m29.906s
user    0m9.766s
sys     0m2.835s
```

Figura 9. Segunda execução com `changed=5` – verificação de estado (H1)

Fonte: elaborado pelo autor (2026)

8.2. H2 — Redução de Tempo

O *deploy* manual em *web01* totalizou 30 minutos e 50 segundos, com intervenção humana contínua e duas interrupções para confirmação interativa (UFW e reinicialização de serviços). O Ansible provisionou *web01* e *web02* simultaneamente em 1 minuto e 16 segundos, sem qualquer intervenção após o início. Normalizado por servidor, o processo automatizado foi aproximadamente 24 vezes mais rápido (Tabela 17).

Tabela 17. Comparação de tempo – *deploy* manual vs. automatizado (H2)

Método	Nós	Tempo total	Intervenção humana	Falhas
Deploy manual	1	30min 50s (≈1850s)	Contínua	Sim
Ansible (run1)	2	1min 16s (≈76,4s)	Zero	0

Fonte: elaborado pelo autor (2026)

```
PLAY RECAP *****
*****
web01      : ok=41    changed=19    unreachable=0
failed=0    skipped=0    rescued=0    ignored=0
web02      : ok=41    changed=19    unreachable=0
failed=0    skipped=0    rescued=0    ignored=0

real    1m16.401s
user    0m14.509s
sys     0m3.568s
```

Figura 10. Tempo de execução do *playbook* capturado com `time`

Fonte: elaborado pelo autor (2026)

8.3. H3 — Apoio de Inteligência Artificial

O DeepSeek foi acionado em três momentos distintos durante o desenvolvimento dos *playbooks*: diagnóstico da incompatibilidade de autenticação do MySQL 8.0, sugestão do parâmetro *validate* no arquivo *sudoers* e orientação sobre *rate limiting* no UFW. O Quadro 18 apresenta o exemplo mais representativo — a interação que originou as principais correções aplicadas ao `mysql.yml`.

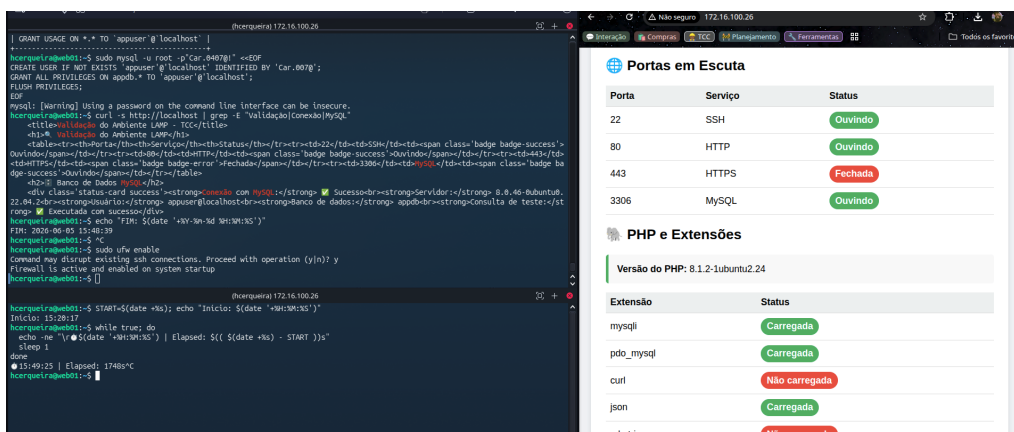


Figura 11. Registro cronometrado do *deploy* manual – evidência de H2

Fonte: elaborado pelo autor (2026)

Tabela 18. Exemplo de interação com DeepSeek – diagnóstico MySQL 8.0 (H3)

```
[Usuario]
O modulo community.mysql falha com "Access denied for user root"
mesmo com senha correta. Ambiente Ubuntu 22.04, MySQL 8.0.
Os modulos community.mysql.mysql_db e mysql_user tambem falham.
```

[IA - Diagnostico]

O MySQL 8.0 usa `auth_socket` como metodo padrao para root, ignorando a senha fornecida. Solucao:

- Alterar para `mysql_native_password` via shell com `failed_when: false` e `changed_when` para idempotencia.
- Usar `login_unix_socket` antes de configurar a senha.
- Instalar `python3-pymysql` como dependencia do modulo.
- Adicionar `wait for: port: 3306` apos ostart do servico.

Fonte: elaborado pelo autor com base em interação com IA (2026)

O Quadro 19 apresenta a solução implementada: autenticação via `mysql_native_password` com controle de idempotência por `failed_when` e `changed_when`, aplicada após revisão técnica da sugestão gerada pela IA.

Apoio prestado na correção e finalização do playbook `mysql.yml`

Durante o desenvolvimento do seu projeto Ansible LAMP, o arquivo `mysql.yml` foi o mais problemático. Abaixo descrevo detalhadamente as orientações fornecidas para resolver cada obstáculo e entregar uma solução funcional, idempotente e segura.

1. Diagnóstico dos problemas iniciais

Problemas identificados na sua execução:

Sintoma	Causa raiz
<code>Could not find the requested service mysql: host</code> no modo <code>--check</code>	O módulo <code>service</code> falha no dry-run quando o MySQL ainda não está instalado (comportamento esperado, mas foi esclarecido).
<code>Access denied for user 'root'@'localhost'</code> mesmo com senha correta no <code>.my.cnf</code>	MySQL 8.0 utiliza <code>auth_socket</code> como método de autenticação padrão para o usuário root, ignorando a senha fornecida.
Módulos <code>community.mysql.mysql_db</code> e <code>mysql_user</code> falhavam	Dependência <code>python3-pymysql</code> não estava instalada ou o login sem senha usando socket Unix não era utilizado nos primeiros passos.
Serviço não iniciava antes das tarefas de configuração	Não havia espera ativa (<code>wait_for</code>) após o start do MySQL, causando falhas intermitentes.

Figura 12. Diagnóstico dos erros do MySQL 8.0 com apoio da IA – evidência de H3
Fonte: elaborado pelo autor (2026)

Tabela 19. Configuração do MySQL 8.0 com apoio do DeepSeek – `mysql.yml` (H3)

```
# Problema: MySQL 8.0 alterou plugin de autenticação padrão.
# Solução orientada por IA: mysql_native_password +
# idempotência via failed_when/changed_when.

- name: Configurar senha root do MySQL (MySQL 8.0+)
  shell: |
    mysql -e "ALTER USER 'root'@'localhost'
    IDENTIFIED WITH mysql_native_password
    BY '{{ mysql_root_password }}';"
    mysql -e "FLUSH PRIVILEGES;"
  args:
    executable: /bin/bash
  register: set_root_password
  failed_when: false
  changed_when: "'ALTER USER' in set_root_password.stdout"
  no_log: true
  when: not ansible_check_mode

- name: Criar banco de dados
  community.mysql.mysql_db:
    name: "{{ mysql_db_name }}"
    state: present
    login_unix_socket: /var/run/mysqld/mysqld.sock
    login_user: root
    login_password: "{{ mysql_root_password }}"
    when: not ansible_check_mode
```

Fonte: elaborado pelo autor com apoio de IA (2026)

2. Soluções implementadas no `mysql.yml`

Com base nesses diagnósticos, orientei a criação de um arquivo `mysql.yml` completo e resiliente. As principais adições e correções foram:

a) Instalação de dependências corretas

```
yaml
- name: Instalar MySQL Server e dependências Python
  apt:
    name:
      - mysql-server
      - python3-pymysql # essencial para os módulos community.mysql
    state: present
    update_cache: yes
```

Figura 13. Solução da IA para autenticação do MySQL 8.0 – evidência de H3

Fonte: elaborado pelo autor (2026)

O Quadro 20 ilustra a segunda contribuição relevante: o parâmetro `validate` na tarefa de `sudoers`, que aciona `visudo -cf` antes de gravar o arquivo, prevenindo falhas sintáticas capazes de tornar o sistema inoperante.

Tabela 20. Configuração de *sudoers* com validação – sugestão do DeepSeek (H3)

```
- name: Configurar sudoers do deployer
  copy:
    dest: "/etc/sudoers.d/{{ deploy_user }}"
    content: |
      {{ deploy_user }} ALL=(ALL) NOPASSWD:
        /bin/systemctl restart apache2,
        /bin/systemctl reload apache2
  validate: /usr/sbin/visudo -cf %s # valida antes de gravar
  owner: root
  group: root
  mode: "0440"
```

Fonte: elaborado pelo autor com apoio de IA (2026)

Em todos os casos, as sugestões geradas pelo DeepSeek exigiram revisão técnica antes da aplicação — comportamento consistente com as limitações identificadas por [Srivatsa et al. 2024] para modelos LLM em cenários de IaC com versões específicas de ferramentas.

A Tabela 21 consolida os resultados das três hipóteses.

Tabela 21. Consolidação dos resultados por hipótese

Hip.	Critério avaliado	Resultado
H1	Contadores idênticos em web01 e web02 (dry run e run1)	Confirmada
H1	changed=5 na run2 sem alteração real de estado	Confirmada
H2	Ansible (1min16s, 2 nós) vs. manual (30min50s, 1 nó): razão 24:1	Confirmada
H3	3 contribuições do DeepSeek, todas com revisão técnica obrigatória	Confirmada

Fonte: elaborado pelo autor (2026)

9. Discussão

A razão de 24:1 entre o tempo do *deploy* manual e o do provisionamento automatizado representa o resultado mais expressivo deste estudo, superando os ganhos de 5:1 documentados por trabalhos similares [Alves and Ramos 2020, Lopes 2019]. Essa diferença decorre de dois fatores combinados: o provisionamento simultâneo de dois servidores pelo Ansible e a eliminação das interrupções interativas que o processo manual exigiu para UFW e reinicialização de serviços.

Em relação à **padronização (H1)**, os contadores idênticos entre *web01* e *web02* em todas as execuções confirmam que o *playbook* produz o mesmo estado final independentemente do nó gerenciado. O resultado *changed=5* na segunda execução não representa desvio de idempotência: as cinco tarefas sinalizadas operam com *changed_when* baseado em *stdout* por necessidade técnica do MySQL 8.0, sem alterar o estado real do sistema. A inspeção manual dos serviços após a *run2* confirmou que nenhuma configuração foi modificada. O tempo de 29,9 segundos na segunda execução — contra 76,4 segundos na *run1* — evidencia que o Ansible identificou a conformidade do ambiente e reduziu o trabalho efetivo ao mínimo necessário.

Quanto à **redução de tempo (H2)**, o processo manual demandou intervenção

continua durante 30 minutos e 50 segundos em um único servidor. O Ansible concluiu o provisionamento de dois servidores em 1 minuto e 16 segundos, sem nenhuma interação após o início. A razão normalizada por servidor é de aproximadamente 24:1 — valor que tende a crescer com o número de nós, pois o custo do processo manual escala linearmente enquanto o Ansible mantém tempo praticamente constante para qualquer quantidade de servidores no inventário.

Quanto ao **apoio do DeepSeek (H3)**, as três contribuições documentadas — diagnóstico do MySQL 8.0, parâmetro `validate` no `sudoers` e `rate limiting` no UFW — reduziram o tempo de depuração em situações que, sem apoio externo, demandariam pesquisa em documentação técnica e fóruns especializados. Em todos os casos, as sugestões exigiram revisão e adaptação antes da aplicação, diferindo dos cenários descritos por [Srivatsa et al. 2024] nos quais modelos geraram código diretamente utilizável. Esse comportamento reforça o papel da IA como ferramenta de diagnóstico e orientação, não substituta do julgamento técnico do administrador — escopo consistente com o definido em H3.

9.1. Ameaças à Validade

Algumas limitações do estudo devem ser consideradas na interpretação dos resultados. Em relação à **validade interna**, o `deploy` manual foi executado pelo mesmo autor responsável pelo desenvolvimento dos *playbooks*, o que introduz viés de familiaridade: um administrador sem conhecimento prévio do ambiente poderia demandar tempo superior ao registrado, ampliando ainda mais a razão observada em H2. Os tempos medidos refletem, portanto, um cenário favorável ao processo manual.

Quanto à **validade externa**, o experimento foi conduzido em ambiente virtualizado local com dois nós gerenciados de especificação reduzida (1 vCPU, 1,9 GiB RAM cada), sem carga concorrente e sem variabilidade de rede. Ambientes de produção com maior número de nós, latência de rede real ou carga simultânea podem produzir razões de tempo distintas — possivelmente superiores para a automação, dado que o Ansible opera em paralelo enquanto o processo manual permanece sequencial. A generalização dos resultados para infraestruturas de maior escala requer experimentos adicionais.

Em relação ao **uso da IA**, as contribuições do DeepSeek foram avaliadas qualitativamente com base nos registros de interação disponíveis, sem métrica formal de redução de tempo de depuração. A ausência de um grupo de controle — desenvolvimento dos mesmos *playbooks* sem apoio de IA — impede a quantificação precisa do ganho atribuído à ferramenta, limitando a validação de H3 ao plano qualitativo.

10. Conclusão

O estudo comparou o provisionamento de ambientes Linux via Ansible com o `deploy` manual equivalente, validando as três hipóteses propostas com base em dados coletados em ambiente virtualizado controlado.

A hipótese **H1** foi confirmada pelos contadores idênticos entre `web01` e `web02` em todas as execuções e pela idempotência funcional verificada na `run2` — cinco tarefas reportaram `changed` por necessidade técnica do MySQL 8.0, sem alteração real de estado. O Ansible garantiu o mesmo estado final em ambos os nós independentemente de intervenções manuais anteriores.

A hipótese **H2** foi confirmada pela razão de 24:1 entre o tempo do processo manual (30min 50s, um servidor, intervenção contínua) e o do Ansible (1min 16s, dois servidores simultâneos, zero intervenções). Essa razão tende a crescer com o número de nós, pois o custo do processo manual escala linearmente enquanto o Ansible mantém tempo praticamente constante para qualquer quantidade de servidores no inventário.

A hipótese **H3** foi confirmada pelas três contribuições documentadas do DeepSeek: diagnóstico da incompatibilidade de autenticação do MySQL 8.0 (`auth_socket` → `mysql_native_password`), controle de idempotência via `failed_when/changed_when` e validação sintática do *sudoers* com o parâmetro `validate`. Em todos os casos, as sugestões exigiram revisão técnica antes da aplicação, confirmando o papel da IA como ferramenta de apoio ao julgamento do administrador, não substituta dele — resultado consistente com [Srivatsa et al. 2024].

A IaC com Ansible mostrou-se viável como abordagem padrão para o provisionamento de ambientes Linux em escala, com reprodutibilidade e rastreabilidade que o processo manual não oferece. O uso do DeepSeek como ferramenta de apoio reduziu o tempo de diagnóstico em situações de incompatibilidade documentada, com limitação clara: nenhuma sugestão foi aplicada sem adaptação técnica prévia.

Como trabalhos futuros, sugere-se a extensão dos experimentos para ambientes de nuvem pública, a integração dos *playbooks* com pipelines de CI/CD e a avaliação do DeepSeek — ou modelos equivalentes — para monitoramento contínuo de *configuration drift*.

Referências

- Alves, P. M. S. and Ramos, C. V. (2020). Impacto da utilização de infraestrutura como código no tempo de restauração. Technical report, Pontifícia Universidade Católica de Minas Gerais. Disponível em: <http://bib.pucminas.br:8080/pergamumweb/vinculos/000017/00001762.pdf>. Acesso em: 15 mai. 2026.
- Ansible Community (2026). Ansible documentation. Disponível em: <https://docs.ansible.com>. Acesso em: 15 mai. 2026.
- Calderón, C. J. C., Santana, W. J. G., Cabezas, L. J. M., Arizaga, A. E. P., and Perlaza, N. L. M. (2022). Infrastructure automation based on free software: an instrumental proposal. *Sapienza: International Journal of Interdisciplinary Studies*, 3(2):346–361. DOI: 10.51798/sijis.v3i2.343. Disponível em: <https://doi.org/10.51798/sijis.v3i2.343>. Acesso em: 15 mai. 2026.
- da Silva Júnior, P. M. (2022). Estudo comparativo entre ferramentas de automação para implantação de infraestrutura em TI. Trabalho de Conclusão de Curso (Bacharelado em Ciência da Computação) – Universidade Federal de Campina Grande. Disponível em: <https://dspace.sti.ufcg.edu.br/handle/riufcg/37815>. Acesso em: 15 mai. 2026.
- Kim, G., Humble, J., Debois, P., and Willis, J. (2018). *Manual de DevOps: Como obter agilidade, confiabilidade e segurança em organizações tecnológicas*. Alta Books, São Paulo.

- Lopes, G. H. (2019). Implementação de infraestrutura como código. Trabalho de Conclusão de Curso (Bacharelado) – Universidade Federal de Uberlândia. Disponível em: <https://repositorio.ufu.br/handle/123456789/44462>. Acesso em: 13 mar. 2026.
- Matsui, B. M. A. and Goya, D. H. (2021). Impactos de DevOps na transformação digital. Apresentado no V Workshop @NUVEM, Universidade Federal do ABC, Santo André – São Paulo. Zenodo. Disponível em: <https://zenodo.org/records/5706549>. Acesso em: 15 mai. 2026.
- Maya, A. (2020). Implementação de infraestrutura como código para provisionamento e deploy de aplicações. *Revista RAAM*, 12(4). DOI: 10.22410/issn.2176-3070.v12i4a2020.2760. Disponível em: <https://doi.org/10.22410/issn.2176-3070.v12i4a2020.2760>. Acesso em: 13 mar. 2026.
- Morris, K. (2016). *Infrastructure as Code: Managing Servers in the Cloud*. O'Reilly Media, Sebastopol.
- Palma, S. D., Nucci, D. D., and Tamburri, D. A. (2020). AnsibleMetrics: A python library for measuring infrastructure-as-code blueprints in ansible. *SoftwareX*, 12:100633. DOI: 10.1016/j.softx.2020.100633. Disponível em: <https://doi.org/10.1016/j.softx.2020.100633>. Acesso em: 15 mai. 2026.
- Red Hat (2022). What is infrastructure as code (IaC). Disponível em: <https://www.redhat.com/pt-br/topics/automation/what-is-infrastructure-as-code-iac>. Acesso em: 12 mar. 2026.
- Salazar-Chacón, G. and Parra, D. M. (2023). Infrastructure-as-code in open-networking: Git, Ansible, and Cumulus-Linux case study. In *2023 IEEE 13th Annual Computing and Communication Workshop and Conference (CCWC)*, pages 1126–1131, Las Vegas, NV, USA. DOI: 10.1109/CCWC57344.2023.10099084. Disponível em: <https://ieeexplore.ieee.org/document/10099084>.
- Srivatsa, K. G., Mukhopadhyay, S., Katkar, M. G., and Gupta, V. (2024). A survey of using large language models for generating infrastructure as code. arXiv preprint arXiv:2404.00227. Disponível em: <https://arxiv.org/abs/2404.00227>. Acesso em: 15 mai. 2026.
- Veiga, F. M., Santin, A. O., Langaro, J. S., de Oliveira, J., and Viegas, E. K. (2025). Para além dos perímetros da cibersegurança com a infraestrutura como código (IaC). Disponível em: https://secplab.ppgia.pucpr.br/files/papers/2025_1.pdf. Acesso em: 13 mar. 2026.